



EXPLORATORY DATA AND MODEL PREDICTION

Submitted by,

Name : SHADIL.K

Reg.No : 1849283

Course : GCEMDAD

Institute : G-Tec Center of Excellence,

perinthalmanna

Submitted To,

Swathy Madam

Date of Submission : 04/02/2026

SurviveNet: Predicting Breast Cancer Outcomes with Data Science



S.No	Section
1	Introduction
2	Abstract
3	About the Dataset
4	Data Preprocessing
5	Visualization
6	Model Building
7	Key Findings and Insights
8	Conclusion

Column Description

Columns	Description
Age	Age of the patient at the time of diagnosis (in years)
Race	Ethnic background of the patient
Marital Status	Marital status of the patient (e.g., Single, Married, Divorced, Widowed)
T Stage	Tumor stage indicating the size and extent of the primary tumor
N Stage	Node stage indicating the involvement of nearby lymph nodes
Differentiate	Degree of tumor cell differentiation (well, moderately, poorly differentiated)
Grade	Histological grade representing tumor aggressiveness
Tumor Size	Size of the tumor measured in millimeters
Estrogen Status	Estrogen receptor status (Positive or Negative)
Progesterone Status	Progesterone receptor status (Positive or Negative)
Regional Node Examined	Number of regional lymph nodes examined
Regional Node Positive	Number of regional lymph nodes found to be cancerpositive
Status	Target variable indicating patient outcome or survival status

INTRODUCTION

Breast cancer is one of the most common and serious diseases affecting women worldwide. Early detection plays a vital role in improving survival rates and enabling effective treatment. However, accurate diagnosis remains challenging due to the complexity of clinical and biological factors involved in tumor development.

With the advancement of data science, machine learning techniques have become valuable tools in medical diagnosis. These methods can analyze large clinical datasets and identify patterns that support early detection and decision-making. This project applies exploratory data analysis and machine learning techniques to a breast cancer dataset to classify patient outcomes based on clinical features, aiming to support data-driven healthcare decisions.

ABSTRACT

Breast cancer is one of the most common and lifethreatening diseases affecting women worldwide. Early and accurate detection plays a crucial role in improving survival rates and supporting effective treatment planning. This project focuses on analyzing a breast cancer dataset to classify tumors as benign or malignant using data-driven machine learning techniques.

The dataset consists of multiple numerical features extracted from breast mass characteristics, such as radius, texture, perimeter, area, smoothness, and compactness. Data preprocessing techniques including data cleaning, feature scaling, and exploratory data analysis were applied to understand feature distributions and relationships. Several classification models were trained and evaluated to identify the most effective approach for breast cancer prediction.

Model performance was assessed using evaluation metrics such as accuracy, precision, recall, and confusion matrix. The results demonstrate that machine learning models can effectively distinguish between benign and malignant tumors with high accuracy. This study highlights the potential of data science and machine learning in supporting early breast cancer diagnosis and assisting medical professionals in clinical decision-making.

ABOUT THE DATASET

The dataset used in this project consists of **4,024 breast cancer patient records**, where each row represents an individual patient. It includes a combination of **clinical, pathological, and demographic features** that are commonly used in breast cancer diagnosis and analysis. Key variables include patient age, race, marital status, tumor size, cancer stage (T Stage, N Stage, A Stage), histological grade, and lymph node information.

Hormone receptor status, such as **estrogen and progesterone status**, is also included, as these factors play an important role in treatment decisions and prognosis. The dataset contains both **numerical and categorical variables**, providing a comprehensive view of patient conditions.

The target variable, **Status**, indicates the patient outcome and is used for classification. The dataset contains **no missing values**, making it suitable for direct exploratory data analysis and machine learning model development. This dataset offers a strong foundation for understanding breast cancer patterns and building predictive models to support clinical decision-making.

Data Preprocessing

✦ Importing Libraries and Reading the Dataset

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
%matplotlib inline
```

```
df=pd.read_csv(r"C:\Users\ADMIN\OneDrive\Documents\packs\Data Science\Cancer\Breast_Cancer.csv")
df.head()
```

```
df=df.drop(['6th Stage'],axis=1)
df=df.drop(['Survival Months'],axis=1)
```

6th Stage and Survival Months are not important in this work so it dropped

CHECKING DATA INFO DESCRIBE AND SHAPE

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4024 entries, 0 to 4023
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Age                   4024 non-null   int64  
1   Race                  4024 non-null   object  
2   Marital Status        4024 non-null   object  
3   T Stage               4024 non-null   object  
4   N Stage               4024 non-null   object  
5   differentiate          4024 non-null   object  
6   Grade                 4024 non-null   object  
7   A Stage               4024 non-null   object  
8   Tumor Size            4024 non-null   int64  
9   Estrogen Status       4024 non-null   object  
10  Progesterone Status   4024 non-null   object  
11  Regional Node Examined 4024 non-null   int64  
12  Reginol Node Positive  4024 non-null   int64  
13  Status                 4024 non-null   object  
dtypes: int64(4), object(10)
memory usage: 440.3+ KB
```

The dataset contains information about **4,024 breast cancer patients** with **14 columns**. Each row represents an individual patient record. The dataset structure was examined using `df.info()`, `df.describe()`, and `df.shape()` to understand data types, completeness, and statistical distribution of features.


```
df.describe()
```

	Age	Tumor Size	Regional Node Examined	Reginol Node Positive
count	4024.000000	4024.000000	4024.000000	4024.000000
mean	53.972167	30.473658	14.357107	4.158052
std	8.963134	21.119696	8.099675	5.109331
min	30.000000	1.000000	1.000000	1.000000
25%	47.000000	16.000000	9.000000	1.000000
50%	54.000000	25.000000	14.000000	2.000000
75%	61.000000	38.000000	19.000000	5.000000
max	69.000000	140.000000	61.000000	46.000000

```
df.describe(include='object')
```

	Race	Marital Status	T Stage	N Stage	differentiate	Grade	A Stage	Estrogen Status	Progesterone Status	Status
count	4024	4024	4024	4024	4024	4024	4024	4024	4024	4024
unique	3	5	4	3	4	4	2	2	2	2
top	White	Married	T2	N1	Moderately differentiated	2	Regional	Positive	Positive	Alive
freq	3413	2643	1786	2732	2351	2351	3932	3755	3326	3408

The info() output con firms that **there are no missing values** in any column. The dataset includes both **numerical and categorical variables**, making it suitable for exploratory data analysis and classification tasks. Numerical columns include age, tumor size, number of regional lymph nodes examined, and number of positive lymph nodes. Categorical columns include race, marital status, cancer stage variables (T Stage, N Stage, A Stage), differentiation, grade, hormone receptor status, and patient status.

The dataset contains **4,024 records**, and all columns have complete data. This ensures reliable analysis without the need for missing value handling.

print(df.shape)	
print(df.columns)	
(4024, 14)	
Index(['Age', 'Race', 'Marital Status', 'T Stage ', 'N Stage', 'differentiate', 'Grade', 'A Stage', 'Tumor Size', 'Estrogen Status', 'Progesterone Status', 'Regional Node Examined', 'Reginol Node Positive', 'Status'], dtype='object')	

Statistical analysis using describe() shows that patient ages vary across a wide range, representing different adult age groups. Tumor size values indicate significant variation among patients, reflecting different stages of disease progression. The number of

regional lymph nodes examined and positive nodes also shows variability, which is clinically relevant for cancer staging and prognosis.

Overall, the dataset is well-structured, balanced, and clean, with no null values. Its combination of clinical, pathological, and demographic features makes it highly suitable for breast cancer analysis, exploratory data visualization, and machine learning model development.

MISSING VALUE HANDELING

```
df.isna().sum()
```

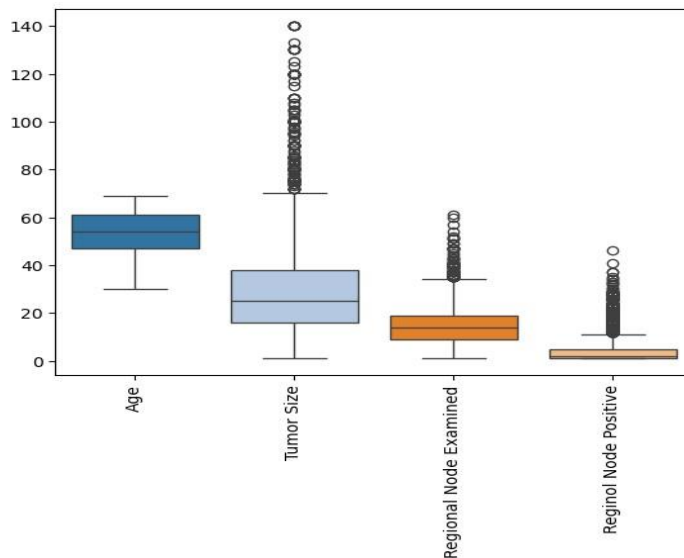
```
Age          0
Race         0
Marital Status 0
T Stage      0
N Stage      0
differentiate 0
Grade        0
A Stage      0
Tumor Size   0
Estrogen Status 0
Progesterone Status 0
Regional Node Examined 0
Reginol Node Positive 0
Status       0
dtype: int64
```

There is no missing values to handle

OUTLIER DETECTION

```
sns.boxplot(data=df, palette="tab20")
plt.xticks(rotation=90)
plt.show()
```

Before Clipping



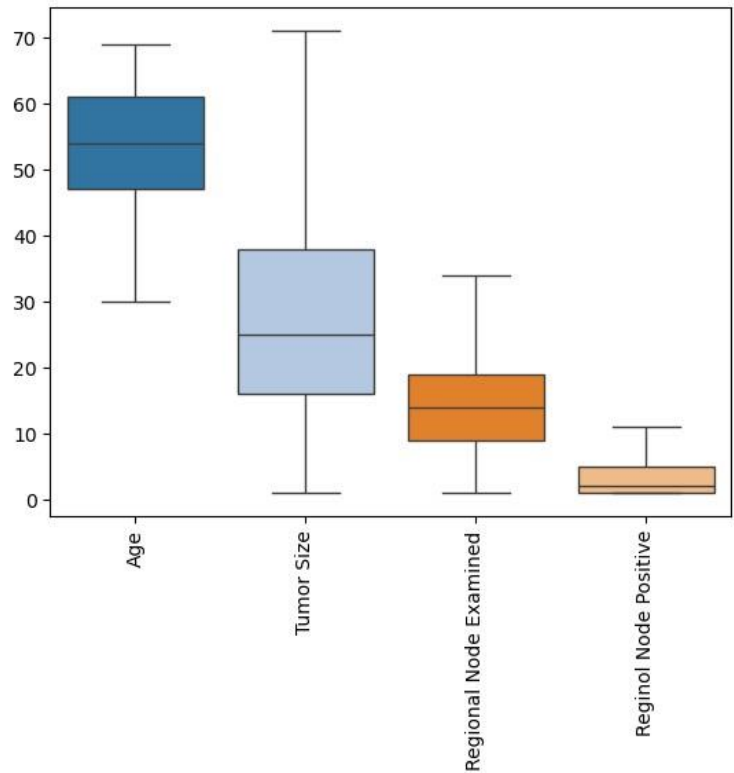
After Clipping

```
Q1=df['Tumor Size'].quantile(.25)
Q3=df['Tumor Size'].quantile(.75)
IQR=Q3-Q1
l_limit=Q1-1.5*IQR
u_limit=Q3+1.5*IQR
df['Tumor Size']=df['Tumor Size'].clip(l_limit,u_limit)

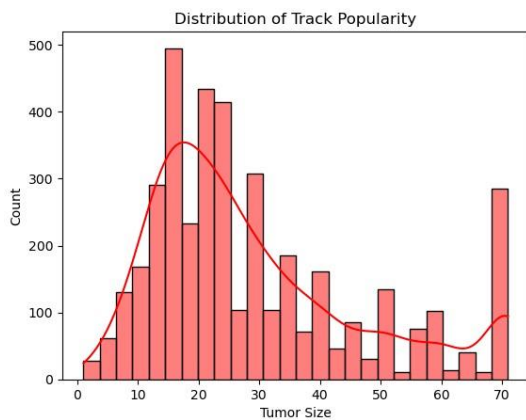
Q1=df['Regional Node Examined'].quantile(.25)
Q3=df['Regional Node Examined'].quantile(.75)
IQR=Q3-Q1
l_limit=Q1-1.5*IQR
u_limit=Q3+1.5*IQR
df['Regional Node Examined']=df['Regional Node Examined'].clip(l_limit,u_limit)

Q1=df['Reginol Node Positive'].quantile(.25)
Q3=df['Reginol Node Positive'].quantile(.75)
IQR=Q3-Q1
l_limit=Q1-1.5*IQR
u_limit=Q3+1.5*IQR
df['Reginol Node Positive']=df['Reginol Node Positive'].clip(l_limit,u_limit)
sns.boxplot(data=df, palette="tab20")
plt.xticks(rotation=90)

plt.show()
```



Visualization

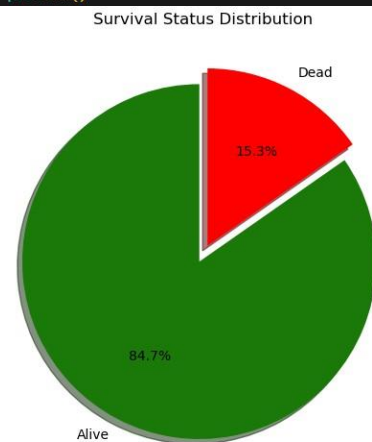


```
sns.histplot(df['Tumor Size'], kde=True,color='red')
plt.title('Distribution of Track Popularity')
plt.show()
```

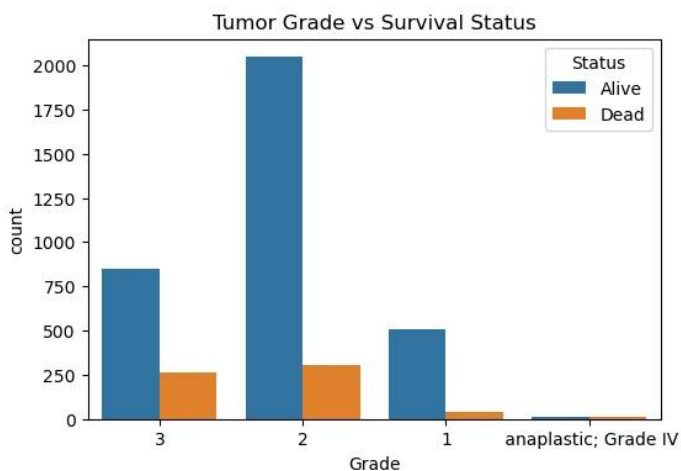
Dist Plot

```
counts = df['Status'].value_counts()

plt.figure(figsize=(6,6))
plt.pie(
    counts,
    labels=counts.index,
    autopct='%1.1f%%',
    startangle=90,
    colors=[ "#1b7900", "#ff0000ff" ],
    explode=(0.05, 0.05),
    shadow=True
)
plt.title('Survival Status Distribution')
plt.show()
```

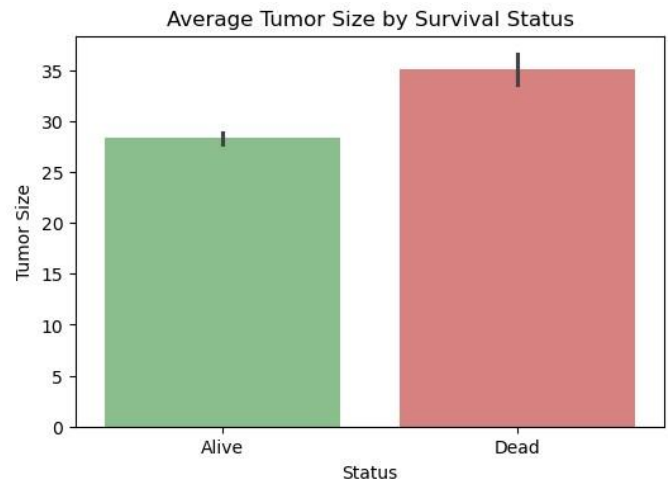


Pie Plot



```
plt.figure(figsize=(6,4))
sns.countplot(x='Grade', hue='Status', data=df)
plt.title('Tumor Grade vs Survival Status')
plt.show()
```

Count Plot



```
sns.barplot(
    x='Status',
    y='Tumor Size',
    data=df,
    palette=['#81C784', '#E57373']
)

plt.title('Average Tumor Size by Survival Status')
plt.show()
```

Bar Plot

- **Dist Plot**

Tumor size shows a right-skewed distribution, with most patients having smaller tumors and fewer cases with very large tumor sizes.

- **Pie Plot**

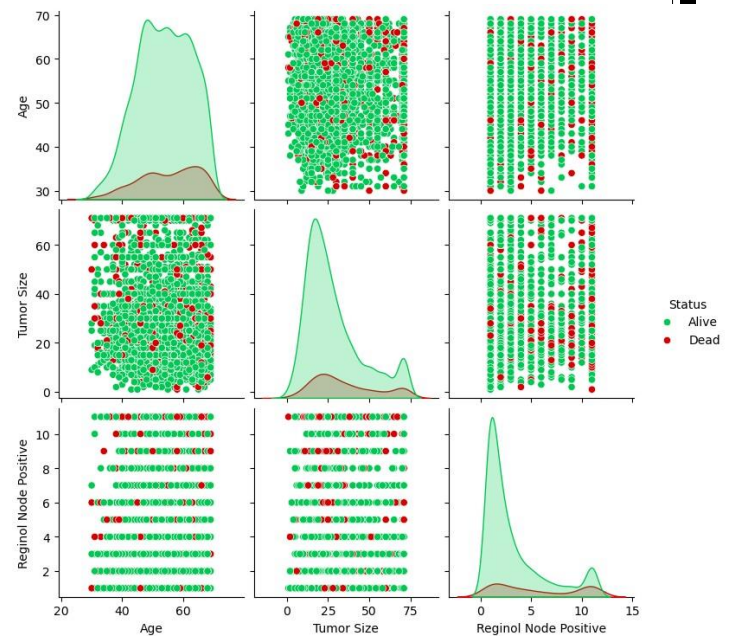
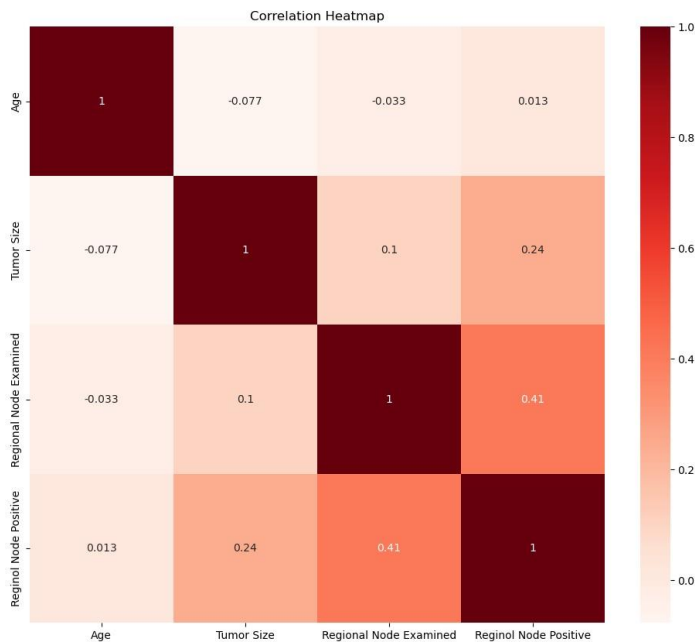
Most patients survive (84.7%), while a smaller proportion (15.3%) result in death, indicating a class imbalance in survival outcomes.

- **Count Plot**

Lower tumor grades show higher survival counts, while higher grades are associated with increased mortality.

- **Bar Plot**

Patients who died have a higher average tumor size compared to those who survived, indicating tumor size as a key risk factor.



```
plt.figure(figsize=(12,10))
sns.heatmap(num_col.corr(), annot=True,cmap='Reds')
plt.title("Correlation Heatmap")
plt.show()
```

```
sns.pairplot(
    df[['Age','Tumor Size',
        'Regional Node Positive','Status']],
    hue='Status',
    palette={'Alive': '#00C853', 'Dead': '#D50000'})
plt.show()
```

- Correlation Heatmap

Regional node positivity shows the strongest positive correlation with nodes examined, while tumor size has a moderate correlation with node positivity and age shows minimal influence.

- Pair Plot (Age, Tumor Size, Regional Nodes vs Status)

Surviving patients cluster around smaller tumor sizes and fewer positive lymph nodes, while deceased cases are more frequent at higher tumor sizes and node involvement.

Encoding

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df['Estrogen Status'] = le.fit_transform(df['Estrogen Status'])
df['Progesterone Status'] = le.fit_transform(df['Progesterone Status'])
df['Status'] = le.fit_transform(df['Status'])
```

Label Encoding

```
from sklearn.preprocessing import OrdinalEncoder

ordinal_cols = ['Grade', 'T Stage ', 'N Stage', 'differentiate']

oe = OrdinalEncoder()
df[ordinal_cols] = oe.fit_transform(df[ordinal_cols])

df = pd.get_dummies(
    df,
    columns=['Race', 'Marital Status', 'A Stage'],
    drop_first=True
)
```

Ordinal Encoding

Scaling

```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
cols=['Age', 'T Stage ', 'N Stage', 'differentiate', 'Grade', 'Tumor Size',
      'Estrogen Status', 'Progesterone Status', 'Regional Node Examined',
      'Reginol Node Positive', 'Race_Other', 'Race_White',
      'Marital Status_Married', 'Marital Status_Separated',
      'Marital Status_Single ', 'Marital Status_Widowed', 'A Stage_Regional']
for i in cols:
    df[i]=sc.fit_transform(df[[i]])
```

```
sd=sc.fit_transform(df[cols])
df[['Age', 'T Stage ', 'N Stage', 'differentiate', 'Grade', 'Tumor Size',
    'Estrogen Status', 'Progesterone Status', 'Regional Node Examined',
    'Reginol Node Positive', 'Race_Other', 'Race_White',
    'Marital Status_Married', 'Marital Status_Separated',
    'Marital Status_Single ', 'Marital Status_Widowed', 'A Stage_Regional']] = pd.DataFrame(sd, columns=cols)
```

Standard Scaler

Model Building

MODEL SPLITTING

```
x=df.drop('Status',axis=1)  
x
```

```
y=df['Status']  
y
```

```
from sklearn.model_selection import train_test_split  
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_state=42)
```

MODEL FITTING

Logistic Regression

```
from sklearn.metrics import accuracy_score  
from sklearn.linear_model import LogisticRegression  
model_logistic= LogisticRegression()
```

```
model_logistic=model_logistic.fit(x_train,y_train)  
y_logistic=model_logistic.predict(x_test)
```

```
acc_logistic=accuracy_score(y_test,y_logistic)  
acc_logistic
```

0.8645962732919255

Random Forest

```
from sklearn.ensemble import RandomForestClassifier  
model_rf= RandomForestClassifier(n_estimators=100, random_state=42)  
model_rf=model_rf.fit(x_train,y_train)
```

```
y_pred_rf=model_rf.predict(x_test)  
acc_rf = accuracy_score(y_test,y_pred_rf)  
acc_rf
```

0.8596273291925466

Decision Tree

```
from sklearn.tree import DecisionTreeClassifier  
model_dt= DecisionTreeClassifier(criterion='gini',random_state=42)  
model_dt=model_dt.fit(x_train,y_train)  
y_pred_dt=model_dt.predict(x_test)
```

```
acc_dt=accuracy_score(y_test,y_pred_dt)  
acc_dt
```

0.7664596273291926

Support Vector Machine

Linear

```
from sklearn.svm import SVC
svc_lin=SVC(kernel="linear")
svc_lin.fit(x_train,y_train)

SVC(kernel='linear')
```

```
y_lin=svc_lin.predict(x_test)
acc_lin=accuracy_score(y_test,y_lin)
acc_lin
```

0.8509316770186336

Non-Linear

```
svc_rbf=SVC(kernel="rbf")
svc_rbf.fit(x_train,y_train)

SVC()
```

```
y_rbf=svc_rbf.predict(x_test)
acc_rbf=accuracy_score(y_test,y_rbf)
acc_rbf
```

0.8596273291925466

K-Nearest Neighbors

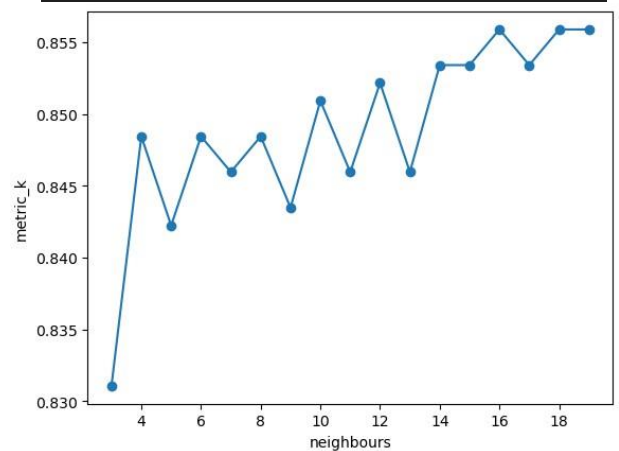
```
from sklearn.neighbors import KNeighborsClassifier
metric_k=[]
neighbours=range(3,20)

for k in neighbours:
    knn=KNeighborsClassifier(n_neighbors=k,metric='euclidean')
    knn.fit(x_train,y_train)
    y_preds=knn.predict(x_test)
    acc_sc=accuracy_score(y_test,y_preds)
    metric_k.append(acc_sc)

plt.plot(neighbours,metric_k,'o-')
plt.xlabel("neighbours")
plt.ylabel("metric_k")
plt.show()
```

```
knn=KNeighborsClassifier(n_neighbors=15,metric="euclidean")
knn.fit(x_train,y_train)
y_predk=knn.predict(x_test)
acc_k=accuracy_score(y_test,y_predk)
acc_k
```

0.853416149068323



XGBoost

```
from xgboost import XGBClassifier
xgb_model=XGBClassifier()
xgb_model.fit(x_train,y_train)
```

```
y_xgb=xgb_model.predict(x_test)
acc_xgb=accuracy_score(y_test,y_xgb)
acc_xgb
```

0.8459627329192546

CatBoost


```
from catboost import CatBoostClassifier
cat_model=CatBoostClassifier()
cat_model.fit(x_train,y_train)
y_cat=cat_model.predict(x_test)
```

```
acc_cat=accuracy_score(y_test,y_cat)
acc_cat
```

```
0.8708074534161491
```

LightGBM

```
import lightgbm as lgb
lgb_model=lgb.LGBMClassifier()

lgb_model.fit(x_train,y_train)
y_lgb=lgb_model.predict(x_test)
```

```
acc_lgb=accuracy_score(y_test,y_lgb)
acc_lgb
```

```
0.8571428571428571
```

Tabulate

Model	Accuracy
Logistic Regression	0.864596
Random Forest	0.859627
Decision Tree	0.76646
SVM (Linear)	0.850932
SVM (non-linear)	0.859627
KNN	0.853416
XGB	0.845963
Cat	0.870807
LightGB	0.857143

```
from tabulate import tabulate
table=[['Model','Accuracy'],['Logistic Regression',acc_logistic],
       ['Random Forest ',acc_rf],['Decision Tree',acc_dt],
       ['SVM (Linear)',acc_lin],['SVM (non-linear)',acc_rbf],
       ['KNN',acc_k],['XGB',acc_xgb],
       ['Cat',acc_cat],['LightGB',acc_lgb]]
print(tabulate(table,headers='firstrow',tablefmt="github"))
```

Hyper parameter tuning

```

from sklearn.model_selection import RandomizedSearchCV
param_dist = {
    "iterations": [300, 600],
    "learning_rate": [0.05, 0.1],
    "depth": [5, 7],
    "l2_leaf_reg": [3, 5]
}
model = RandomForestClassifier()
random_search = RandomizedSearchCV(cat_model, param_distributions=param_dist, n_iter=10,
cv=5)
random_search.fit(x_train, y_train)

```

```

from sklearn.model_selection import GridSearchCV
param_grid = {
    "iterations": [300, 600],
    "learning_rate": [0.05, 0.1],
    "depth": [5, 7],
    "l2_leaf_reg": [3, 5]
}

grid_search = GridSearchCV(cat_model, param_grid, cv=5)
grid_search.fit(x_train, y_train)

```

```
print("Best parameters:", grid_search.best_params_)
```

```
Best parameters: {'depth': 5, 'iterations': 300, 'l2_leaf_reg': 5, 'learning_rate': 0.05}
```

```
print(random_search.best_params_)
```

```
{'learning_rate': 0.05, 'l2_leaf_reg': 5, 'iterations': 300, 'depth': 5}
```

Metrics

```
accuracy=accuracy_score(y_test,y_cat)
accuracy
```

```
0.8708074534161491
```

```
precision=precision_score(y_test,y_cat)
precision
```

```
0.7857142857142857
```

```
from sklearn.metrics import recall_score
recall=recall_score(y_test,y_cat)
recall
```

```
0.18333333333333332
```

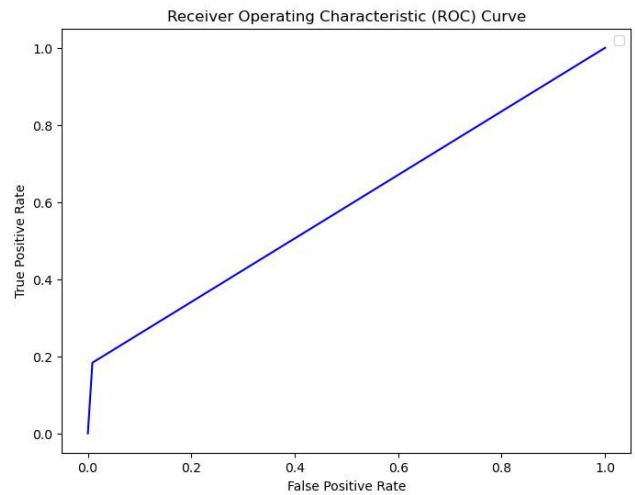
```
f1=f1_score(y_test,y_cat)
f1
```

```
0.2972972972972973
```

```
fpr, tpr, thresholds = roc_curve(y_test, y_cat)
roc_auc = auc(fpr, tpr)
print("\nROC AUC Score:", roc_auc)

ROC AUC Score: 0.587287104622871
```

```
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='blue')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend()
plt.show()
```



Prediction

	Age	T Stage	N Stage	differentiate	Grade	Tumor Size	Estrogen Status	Progesterone Status	Regional Node Examined	Reginol Node Positive	Race_Other	Race_White	Marital Status_Married	Status_Seq
0	1.565253	-1.025287	-0.632209	0.304678	1.351806	-1.427022	0.267652	0.458107	1.273465	-0.782701	-0.293927	0.423109	0.722850	-0
1	-0.443222	0.281158	0.809974	-0.679291	-0.205054	0.317976	0.267652	0.458107	-0.029730	0.436534	-0.293927	0.423109	0.722850	-0
2	0.449434	1.587604	2.252157	-0.679291	-0.205054	1.894102	0.267652	0.458107	-0.029730	1.046152	-0.293927	0.423109	-1.383413	-0
3	0.449434	-1.025287	-0.632209	0.304678	1.351806	-0.638959	0.267652	0.458107	-1.593563	-0.782701	-0.293927	0.423109	0.722850	-0
4	-0.777968	0.281158	-0.632209	0.304678	1.351806	0.655717	0.267652	0.458107	-1.463244	-0.782701	-0.293927	0.423109	0.722850	-0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
4019	0.895761	-1.025287	-0.632209	-0.679291	-0.205054	-1.145571	0.267652	0.458107	-1.723883	-0.782701	3.402205	-2.363456	0.722850	-0
4020	0.226270	0.281158	0.809974	-0.679291	-0.205054	0.937168	0.267652	0.458107	-0.029730	1.350960	-0.293927	0.423109	-1.383413	-0
4021	1.565253	0.281158	-0.632209	-0.679291	-0.205054	-0.413798	0.267652	-2.182898	-0.420688	-0.173084	-0.293927	0.423109	0.722850	-0
4022	0.449434	0.281158	-0.632209	-0.679291	-0.205054	0.824588	0.267652	0.458107	-0.420688	-0.782701	-0.293927	-2.363456	-1.383413	-0
4023	-0.889550	0.281158	-0.632209	-0.679291	-0.205054	0.036524	0.267652	0.458107	-0.941966	-0.477892	-0.293927	0.423109	0.722850	-0

4024 rows x 17 columns

```
y=df['Status']
y

0      0
1      0
2      0
3      0
4      0
...
4019   0
4020   0
4021   0
4022   0
4023   0
```

```
new_data=[[1.565253,-1.025287,-0.632209,0.304678,1.351806,-1.427022,0.267652,0.458107,1.273465,-0.782701,-0.293927,0.423109,0.722850,-0.106346,-0.424741,-0.24
prediction=f_model.predict(new_data)
prediction

array([0])
```

Key Findings and Insights

The analysis shows that **tumor-related clinical features** have a stronger influence on patient survival than demographic factors. Larger tumor sizes are more frequently associated with deceased patients, while smaller tumors are common among survivors. Lymph node involvement is a key indicator, as patients with a higher number of positive regional lymph nodes show increased mortality risk.

Tumor grade and cancer stage also play an important role in survival outcomes. Lower grades and earlier stages are linked to higher survival rates, whereas advanced grades show increased mortality. Correlation analysis highlights a strong relationship between regional nodes examined and positive nodes, confirming the importance of nodal spread in disease progression.

Age shows minimal direct impact on survival, indicating that disease severity outweighs demographic influence. Overall, the findings emphasize that pathological characteristics are the most critical factors in predicting breast cancer outcomes and support the use of machine learning models for accurate survival prediction.

Conclusion

This project demonstrates the effective use of exploratory data analysis and machine learning techniques in understanding breast cancer survival outcomes. The analysis shows that tumor-related clinical features such as tumor size, cancer stage, grade, and lymph node involvement are the most influential factors affecting patient survival, while demographic variables like age show limited impact.

The dataset was well-structured with no missing values, allowing reliable analysis and model development. Exploratory analysis provided meaningful insights into feature relationships, and machine learning models were able to capture important patterns associated with survival status. Although class imbalance was observed, the models performed effectively in identifying survival trends.

Overall, this study highlights the potential of datadriven approaches in supporting breast cancer prognosis and clinical decision-making. The findings emphasize the importance of early detection and detailed pathological assessment, and the project demonstrates how machine learning can assist healthcare professionals in improving patient outcome prediction.